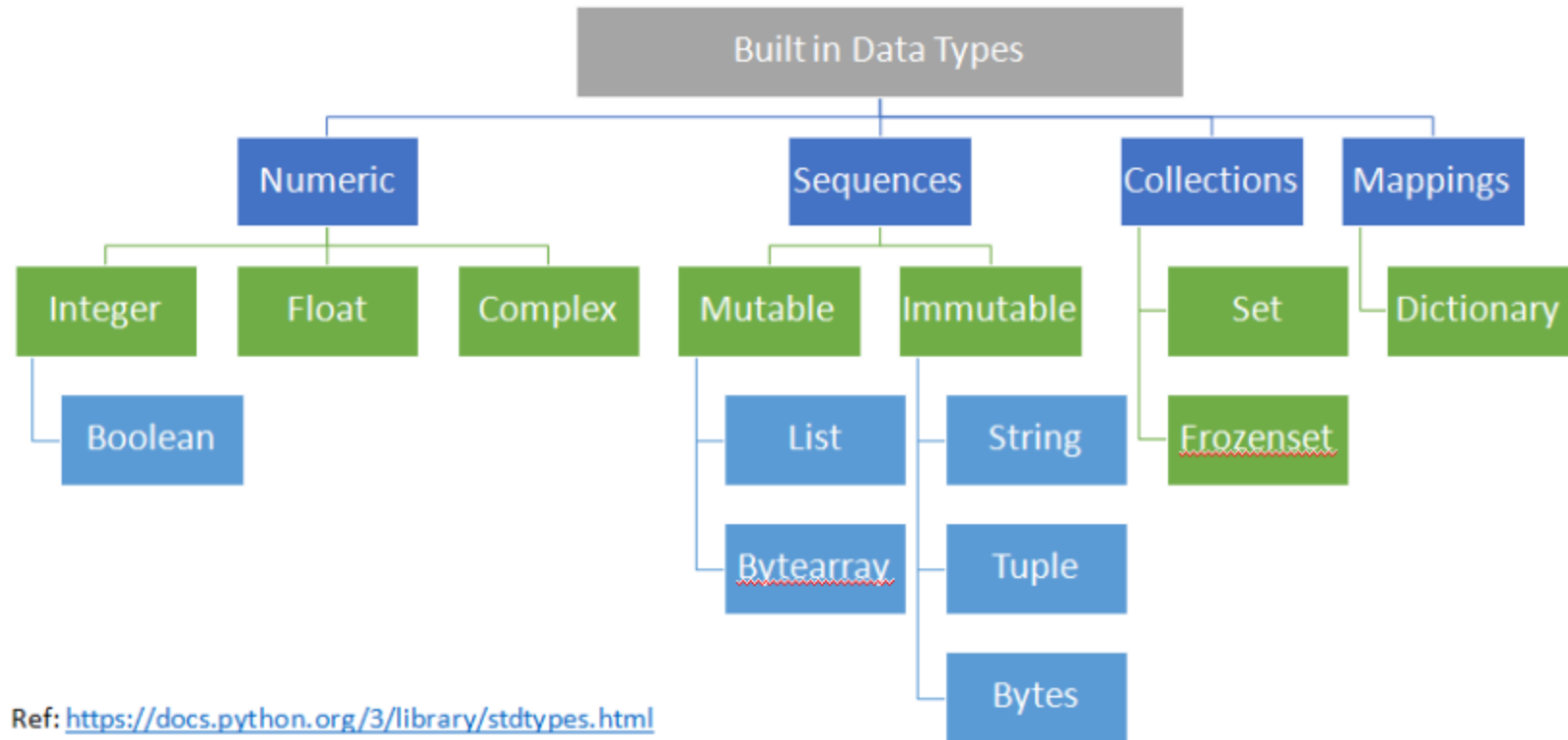


Python Built-in Data type Hierarchy



what is need of data types ?

all values in memory are binary

so when we store 65 as integer or 'A', memory stores it as same binary format

distinction is done based on the data type

data type helps us understand what is type of the data in the memory, it helps us perform proper operations on the

data

ways of assigning data types

- Static typing

at compile time --> like C, C++, java ...

- dynamic typing

at run time [>]--> like python, java script, shell script ,..

↳ 1 cell hidden

type conversion function in python

there are ways to convert between data types

int()

float()

str()

bool()

and many more functions....

↳ 3 cells hidden

✓ Mutable and Immutable Data types

Data type	Mutable / Immutable	Size
<code>int</code>	Immutable	Dynamic
<code>float</code>	Immutable	Fix
<code>complex</code>	Immutable	Fix
<code>bool</code>	Immutable	Fix
<code>str</code> (string)	Immutable	Fix
<code>tuple</code>	Immutable (*Special case)	Fix
<code>list</code>	Mutable	Dynamic
<code>set</code>	Mutable	Dynamic
<code>dict</code>	Mutable	Dynamic
<code>bytes</code>	Immutable	Fix
<code>bytearray</code>	Mutable	Dynamic
<code>frozenset</code>	Immutable	Fix

What is immutable data type ?

Immutable data type : when we assign value then it **cannot be changed**.

In case there is any modification then new memory location is allocated.

Example: below diagram shows program temp1.py

It is creating variable `i` with value 10. `i` is object reference on the stack. 10 is allocated memory on heap. When program assigns new value 20, then new memory location in heap is allocated.

NOTE: Run code given below and you will see two different heap address (id) for the same variable, because we have changed its value in the program

changed its value in the program.

↳ 2 cells hidden

What is Mutable Data type?

When we make changes to the object then original object is changed.

↳ 1 cell hidden

>

int data type in python

- dynamic memory allocation
- there interning done in int
- size of int is unlimited in python, so very simply supports scientific computing
- basic structure of int take 24 bytes and then 4 bytes are allocated for range between 0 to $2^{30} - 1$ then 4 bytes will be added for every 2^{30} range

↳ 5 cells hidden

Similarly float, bool, complex are other numeric data types

- they all are immutable
- they are all fixed in size
- there is no interning done for float / complex
- bool is interned because it is sub type of int

↳ 3 cells hidden

List data type

- one of the most flexible data type
- one of the most used data type by programmers
- like array which allows any data type to be stored in it
- size is dynamic --> it can grow / shrink based on need
- list stores multiple values in a sequence

Ex

```
l1 = [10,20,30]
```

- **every element in list is a reference / place holder**
- list never stores values, it only stores references

↳ 24 cells hidden

✓ Common Simple Questions on list

```
1 # Q Write a Python program to sum all the items in a list.
2 l1=[10,4,6,8,12] # --> make total of all values and print it
3 total =0
4 #how we access all elements one by one ? --> use for loop
5 for i in range(len(l1)): #len(l1) = 5 , range(5) --> 0 1 2 3 4 --> i takes these values
6     print(l1[i]) #l1[0] -> 10,
7     total = total + l1[i]
```

```
8 print(total)
9 print(sum(l1))
```

```
10
4
6
8
12
40
40
```

```
1 ## Q Write a Python program to multiply all the items in a list.
2 l1=[10,4,6,8,12] # --> make total of all values and print it
3 total =1
4 #how we access all elements one by one ? --> use for loop
5 for i in range(len(l1)): #len(l1) = 5 , range(5) --> 0 1 2 3 4 --> i takes these values
6   print(l1[i]) #l1[0] -> 10,
7   total = total * l1[i]
8 print(total)
```

```
10
4
6
8
12
23040
```

```
1 #Write a Python program to get the largest number from a list.
2 l1=[10,4,6,8,12]
3 print("using max=",max(l1)) #method 1
4 #method 2
5 large = l1[0]
6 for val in l1:
7   if val > large:
8     large =val
9 print("using for loop=",large)
```

```
using max= 12
using for loop= 12
```

```
using for loop= 12
```

```
1 #Write a Python program to get the smallest /min number from a list.
2 l1=[10,4,6,8,12]
3 print("using min=",min(l1)) #method 1
4 #method 2
5 small = l1[0]
6 for val in l1:
7     if val < small:
8         small =val
9 print("using for loop=",small)
```

```
using min= 4
using for loop= 4
```

```
1 # 7. Write a Python program to remove duplicates from a list.
2 # Use another list to store unique values.
3 l1=[23,67,98,34,34,23,1,23,1,1,1]
4 #method1
5 l2=[] #empty list
6 for i in range(len(l1)):
7     if l1[i] in l2: # in operator checks if value is present in given list/sequence
8         pass
9     else:
10         l2.append(l1[i])
11 print(l1)
12 print(l2) # w/o duplicates
13 #method 2 --> use set() --> keep only unique values
14 l1=[23,67,98,34,34,23,1,23,1,1,1]
15 l2 = list(set(l1))
16 print(l1)
17 print(l2)
18 #method 3
19 l1=[23,67,98,34,34,23,1,23,1,1,1]
20 l2 = l1.copy()
21 for val in l1:
22     c = l2.count(val)
23     while c > 1:
```



```

23     while c < len(l1):
24         l2.remove(val)
25         c=c-1
26 print(l1)
27 print(l2)

```

```

[23, 67, 98, 34, 34, 23, 1, 23, 1, 1, 1]
[23, 67, 98, 34, 1]
[23, 67, 98, 34, 34, 23, 1, 23, 1, 1, 1]
[1, 98, 67, 34, 23]
[1, 1, 1]
[1]

```

```

1 """
2 # 19. Write a Python program to calculate the difference between the two lists.
3 ex.
4 l1=[1,4,7,8]
5 l2=[2,5,7,23]
6 ans => total difference is 17
7 explanation:
8 first element diff 1
9 second element diff 1
10 third element diff 0
11 fourth element diff 15"""
12 l1=[1,4,7,8]
13 l2=[2,5,7,23]
14 #method 1
15 total_diff=0
16 for i in range(len(l1)):#0,1,2,3
17     total_diff = total_diff + abs(l1[i] - l2[i])
18 print("total_diff =",total_diff)
19 #method 2
20 print("total_diff =", abs(sum(l1)-sum(l2)))

```

```

total_diff = 17
total_diff = 17

```

```

1 #take two lists

```

```

2 #l1=[1,2,3,4]
3 #l2 = [10,20,30,40]
4 #print in format --> 1, 40    2,30    3,20    4,10
5 #hint :: how to reverse a list
6 l1=[1,2,3,4]
7 l2 = [10,20,30,40]
8 l2.reverse()
9 for i in range(len(l1)):
10     print(l1[i],l2[i])

```

```

1 40
2 30
3 20
4 10

```

```

1 """
2 4. Print two lists in the following order
3 list1 = [5, 6,7]
4 list2 = [0, 1]
5 output:
6 50,51,60,61,70,71
7 """
8 l1 = [5, 6,7]
9 l2 = [0, 1]
10 for i in range(len(l1)):
11     for j in range(len(l2)):
12         print(l1[i],l2[j])
13
14 #Advanced code
15 l1 = [5, 6,7]
16 l2 = [0, 1]
17 for i in range(len(l1)):
18     for j in range(len(l2)):
19         print(l1[i],l2[j],sep="",end=",")
20

```

```

5 0
5 1
~ ~

```

```
6 0
6 1
7 0
7 1
50,51,60,61,70,71,
```

```
1 """
2 9. Given a Python list, find value 20 in the list, and if it is present,
3 replace it with 200. Only update the first occurrence of a value
4 list1 = [5, 10, 15, 20, 25, 50, 20]
5 output:
6 list1 = [5, 10, 15, 200, 25, 50, 20]
7 """
8 list1 = [5, 10, 15, 20, 25, 50, 20]
9 for i in range(len(list1)):
10     if list1[i] ==20:
11         list1[i]=200
12         break
13
14 print(list1)
```

```
[5, 10, 15, 200, 25, 50, 20]
```

✓ How to remove elements inside a for loop

```
1 """
2 10. Given a Python list, remove all occurrence of 20 from the list
3 list1 = [5, 20, 15, 20, 25, 50, 20]
4 output:
5 [5, 15, 25, 50]
6 """
7 #method 1 :: create a shallow copy and remove from shallow copy
8 list1 = [5, 20, 15, 20, 25, 50, 20]
9 list2= list1.copy()
10 for i in range(len(list1)): #0,1,2,3,4,5,6
11     if list1[i] == 20: # this is going to give ERROR
12         del list1[i]
```

```

12     list2.remove(20) #-> remove from shallow copy
13 print(list2)
14
15 #method2
16 list1 = [5, 20, 15, 20, 25, 50, 20]
17 list2 =[]
18 for i in range(len(list1)):
19     if list1[i] != 20:
20         list2.append(list1[i])
21 print(list2)
22
23 ### ERROR code --> be careful!!!
24 """
25 list1 = [5, 20, 15, 20, 25, 50, 20]
26 for i in range(len(list1)): #0,1,2,3,4,5,6
27     if list1[i] == 20: # this is going to give ERROR
28         list1.remove(20) #-> remove changes size of the list
29 print(list1)
30 """
31 print("Completed!")

```

```
[5, 15, 25, 50]
```

```
[5, 15, 25, 50]
```

```
Completed!
```

✓ How to remove all elements from a list ?

```

1 l1 = [10,20,30,40]
2 #method1
3 l1.clear()
4 print(l1)
5 #method 2
6 l1 = [10,20,30,40]
7 for i in range(len(l1)):
8     del l1[0]
9 print(l1)
10 #method 3

```

```
-- ..... --  
11 l1 = [10,20,30,40]  
12 for i in range(len(l1)):  
13     l1.pop()  
14 print(l1)
```

```
[]  
[]  
[]
```

✓ String data type

every character / list of characters is a sting in python

whenever there is a string --> it is list of characters

string maintains list of references to every character

strings are immutable --> means value once assigned cannot be changed. if we change the value then new memory is allocated!!

we cannot modify a single character from a string !! --> because immutable!

in strings all characters are interned, so no extra memory every time --> all characters like A-Z, a-z, and 0-9 are allocated fixed cache and they are reused everytime.

```
1 s1 = "DYPERF"  
2 print(s1,type(s1))  
3 s2 = "D"  
4 print(s2,type(s2))
```

```
DYPERF <class 'str'>  
D <class 'str'>
```

```
1 # string is immutable  
2
```

```
3 s1 = "ABC"
4 print(s1,id(s1))
5 s1 = "ABCD"
6 print(s1,id(s1))
7 #cannot replace one char from string --> ERROR
8 #s1[1]="G" #replce B by G --> NOT ALLOWED --> ERROR
```

```
ABC 11064384
ABCD 135913589756272
```

How two strings / lists / tuples are compared directly in python ?

python uses **lexicographical ordering**

✓ Common operations on string

```
1 # create a string -->
2 # 1. direct assignment
3 s1 = "AI with NLP"
4 print(s1)
5 #2.use str() to convert other data type to string
6 s2=str(10.10)
7 print(s2,type(s2))
```

```
AI with NLP
10.1 <class 'str'>
```

```
1 # Seperate characters from a string --> use list()
2 s1= "DYPERF"
3 print(s1, list(s1))
4 # using for loop
5 for c in s1:
6     print(c)
```

```
DYPERF ['D', 'Y', 'P', 'E', 'R', 'F']
D
Y
P
E
R
F
```

```
1 # Take a number from user and print sum of all its digits
2 #-->usage --> calculate amstrong no, all digit multiplication
3 # any code which uses digits of nos seperately
4
5 # simple C language style approach
6 no = int(input("Enter a no"))
7 total = 0
8 while no >0:
9     digit = no % 10
10    print(digit)
11    total = total + digit
12    no = no //10
13 print(total)
14 # method 2 python way
15 no = input("Enter a no") #keep the input as string
16 digit_no = list(no) # take the string and convert to list of digits!
17 total =0
18 for digit in digit_no:# digit is string
19     total = total + int(digit)
20 print(total)
```

```
Enter a no1234
4
3
2
1
10
Enter a no1234
10
```

```
1 # take a string
2 # count occurrence of substring w/o overlap
3
4 s1 = "112233112233"
5 sub = "11" # 11 is coming 2 times ...
6
7 print(s1.count(sub)) # count doesn't consider overlap
```

2

```
1 # take a string
2 # count occurrence of substring with overlap
3 s1= "1111"
4 sub="11" # --> repeatition --> 3 with overlap
5 count=0
6 for i in range(len(s1)):
7     if s1[i:(i+len(sub))] == sub: # considers overlap also
8         count= count +1
9 print(count)
```

3

✓ Find the substring in a string and get its position

- find() or index()

```
1 s1="i am currently in india. india is a wonderful country to live."
2 #find position of india in the string from left hand side ...
3 print(s1.find("india"))
4 print(s1.index("india"))
5 #find position of india in the string from RIGHT hand side ...
6 print(s1.rfind("india"))
7 print(s1.rindex("india"))
8 # we can write for loop to do this
```

18


```
18  
25  
25
```

- ✓ How to check if given string all characters are digits / alphabates / alpha numeric ?

```
1 s1="123"  
2 print(s1.isdigit(),s1.isalpha(),s1.isalnum())  
3 s1="ABC"  
4 print(s1.isdigit(),s1.isalpha(),s1.isalnum())  
5 s1=" %%&& BNM"  
6 print(s1.isdigit(),s1.isalpha(),s1.isalnum())
```

```
True False True  
False True True  
False False False
```

- ✓ Convert given string to lower case / upper case ?

```
1 s1="ABCdef"  
2 print(s1.lower()) # it returns a copy with all lowercase  
3 print(s1.upper())# it returns a copy with all uppercise
```

```
abcdef  
ABCDEF
```

- ✓ **split and join functions in strings**

split function

take a string then splits it using a seperator and returns list of strings

```
1 ip_addr="10.10.10.10"
2 print(ip_addr.split("."))
```

```
['10', '10', '10', '10']
```

```
1 vals = "1,2,3,4,5"
2 print(vals.split(","))
```

```
['1', '2', '3', '4', '5']
```

join function

take **list of strings**

join all the strings using given separator

exact opposite of split

```
1 l1 = ['10', '10', '10', '10']
2 print(".".join(l1))
3 l2=['1', '2', '3', '4', '5'] # all are strings
4 print(",".join(l2))
```

```
10.10.10.10
1,2,3,4,5
```

Use of split & join ...

1. take multiple values input from a user

↳ 2 cells hidden

✓ Slicing in string

SAME as slicing in lists !!!

Home work

```
s1 = "today is a wonderful day!"
```

```
s1[-4:-10]
```

```
s1[-9:-2]
```

```
s1[0:6:2]
```

```
s1[2:5]
```

```
s1[::-1]
```

```
s1[2:-2]
```

```
s1[-5:24]
```

```
s1[::-2]
```

```
s1[::2]
```

✓ Tuple data Type

- tuple is like a list but immutable (with a special case)
- tuple can contain any data type
- when tuple contains mutable data type (like list / set / dictionary) --> then values inside that mutable object can be changed!!
- tuple slicing --> same as list / string
- comparison of two tuples --> using lexicographical ordering

```
1 t1=(1,2,3) # create tuple using round brackets
2 # Error when try to modify immutable object
3 #t1[0]=100 # Noooo not allowed !!!
4
5 # Allowed to change values inside a mutable object!!
6 t2 = (1,2,[100,200,300])
7 print(t2)
8 t2[2][1]=999
9 print(t2)
```

```
(1, 2, [100, 200, 300])
(1, 2, [100, 999, 300])
```

```
1 # for loop on tuple
2 t1=(9,5,3,2)
3 for i in range(len(t1)):
4     print(t1[i])
```

```
9
5
3
2
```

✓ Application of tuple

- to return multiple values from a function

python function can return multiple value because it converts all values into a single tuple !!!

Ex

divmod --> it returns two values, division and modulus of two numbers

enumerate --> it returns two values, index and element from a list

- to swap multiple numbers / variables at a time

```
1 a=10
2 b=20
3 print(a,b)
4 a,b = b,a # swap a and b --> internally python creates a tuple here
5 # tuple stores the values on right side temporarily!!
6 print(a,b)
7 a=10
8 b=20
9 c=30
10 print(a,b,c)
11 a,b,c = c,a,b
12 print(a,b,c)
```

```
10 20
20 10
10 20 30
30 10 20
```

✓ How to swap two numbers ?

1. using direct assignment in python

```
a,b = b,a
```

2. using addition and subtraction

```
a = a+b
```

```
b = a-b
```

```
a = a-b
```

3. using temporary variable

```
t=a
```

```
a=b
```

b=t

4. using XOR

```
1 # divmod
2 a=105
3 b=10
4 print(divmod(a,b)) # o/p is tuple of two values
```

(10, 5)

✓ List Vs Tuple

Python Lists	Python Tuples
List are mutable	Tuples are immutable
Iterations are time-consuming	Iterations are comparatively Faster
Inserting and deleting items is easier with a list.	Read-only access to elements is best accomplished with a tuple data type.
Lists consume more memory	Tuple consumes less memory than the list
Lists are slower because there dynamic	Tuples tend to be faster because of immutable static nature
Lists are NOT Hashable because they are mutable	Tuples are Hashable
Lists have several built-in methods.	A tuple does not have many built-in methods because of immutability
A unexpected change or error is more likely to occur in a list.	In a tuple, changes and errors don't usually occur because of immutability.
Similarity in Lists and Tuples	
Both can have +ve and -ve indexing	
Both can be iterated in for loop	
In both, elements can be read using index directly, index starts from 0	
Both can store elements of any data type in them	

Both can store elements of any data type in them

✓ List Vs Dictionary

List	Dictionary
List is a collection of elements of any data type	Dictionary is a hashed structure of key and value pairs.
Search operation in list is linear, means it is very slow.	Search operation in dictionary is very fast based on hashing
Search operation in list takes $O(n)$ time where n is number of elements	Search in dictionary is $O(1)$ means fixed time for any number of elements
List is created by placing elements in <code>[]</code> separated by commas <code>,</code>	Dictionary is created by placing elements in <code>{ }</code> as <code>"key": "value"</code> , each key value pair is separated by commas <code>,</code>
The indices of list are integers starting from 0.	Indices of dictionary are keys. Keys of dictionary can be of any immutable data type.
The elements are accessed via <u>+ve</u> and <u>-ve</u> indices.	The elements are accessed via key-values.
Slicing can be done on lists using indexes	Dictionary cannot be sliced
The order of the elements entered are maintained.	After Python 3.6, dictionaries maintain order

✓ List vs Tuple vs Set vs Dictionary

List	Tuple	Set	Dictionary
List is a non-homogeneous data structure that stores the elements in single row and	Tuple is also a non-homogeneous data structure that stores single row and multiple rows and columns	Set data structure is also non-homogeneous data structure but stores in single row	Dictionary is also a non-homogeneous data structure which stores key value pairs

multiple rows and columns			
List can be represented by []	Tuple can be represented by ()	Set can be represented by { }	Dictionary can be represented by { }
List allows duplicate elements	Tuple allows duplicate elements	Set will not allow duplicate elements	Set will not allow duplicate elements and dictionary doesn't allow duplicate keys.
List can have any objects as elements	Tuple can have any objects as elements	Set can have ONLY <u>hashable</u> objects as elements	In Dictionary, keys can have ONLY <u>hashable</u> objects as elements In Dictionary values can have any objects
Example: [1, 2, 3, 4, 5]	Example: (1, 2, 3, 4, 5)	Example: {1, 2, 3, 4, 5}	Example: {1:2, 3:4, 100: 34}
List can be created using list() function	Tuple can be created using <u>tuple()</u> function.	Set can be created using set() function	Dictionary can be created using <u>dict()</u> function.
List is mutable <u>i.e</u> we can make any changes in list.	Tuple is immutable <u>i.e</u> we <u>can not</u> make any changes in tuple	Set is mutable <u>i.e</u> we can make any changes in set. But elements are not duplicated.	Dictionary is mutable. But Keys are not duplicated.
List is ordered	Tuple is ordered	Set is unordered	Dictionary is ordered (Python 3.7 and above)
Creating an empty list	Creating an empty Tuple	Creating a set	Creating an empty dictionary

<code>l=[]</code>	<code>t=()</code>	<code>a=set()</code> <code>b=set(a)</code>	<code>d={}</code>
-------------------	-------------------	---	-------------------

✓ Types of Copy in python

- shallow copy

it copies the reference from the list

here when we change immutable elements in copy then original remains as it is

here when we change mutable elements then they are changed in both original and copy

we use `copy()` function or slicing for creating shallow copy of list

- copy by assignment

here only reference of the list in stack is copied

so any changes in copy will affect the original

no extra memory on heap is allocated here

such copy is created when we use assignment operator

ex `l1=[10,20,30]`

`l2 = l1 # --> copy by assignment`

- deep copy (using copy library)

here all mutable objects are allocated separate memory on heap

so, **any changes** mutable / immutable to copy **will NOT affect the original**

it requires very high amount of memory on heap because all mutable objects are allocated new memory

```

1 # shallow copy
2 l1 = [10,20,30,[400,500]]
3 # index 0 --> 10, index 1 --> 20 index 2 --> 30, index 3--> [400,500]
4 l2 = l1.copy() # shallow copy
5 l1[0] = 99
6 # l2 is not affected
7 print(l1,l2)
8 l1[3][0]=999 #update 400
9 #l2 will be affected
10 print(l1,l2)

```

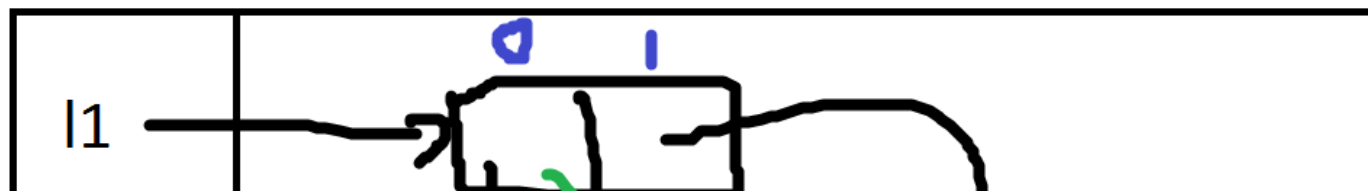
```

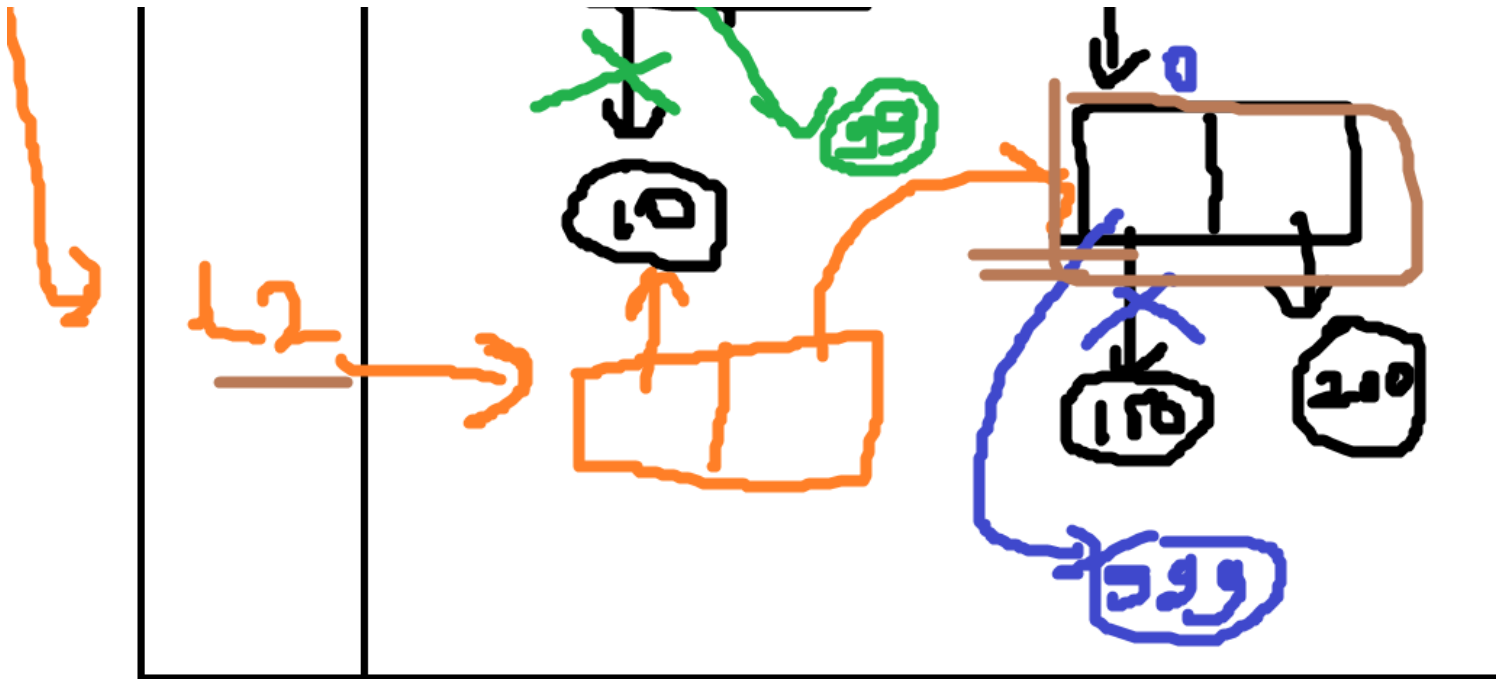
[99, 20, 30, [400, 500]] [10, 20, 30, [400, 500]]
[99, 20, 30, [999, 500]] [10, 20, 30, [999, 500]]

```

Shallow copy

`l1 = [10,[100,200]]`
`l2 = l1.copy()`
`l1[0] = 99` → *l2 not affected*
`l1[1][0] = 999` → *l2 affected*
S H





Copy by assignment

```
l1=[10,[100,200]]
```

```
l2 = l1
```

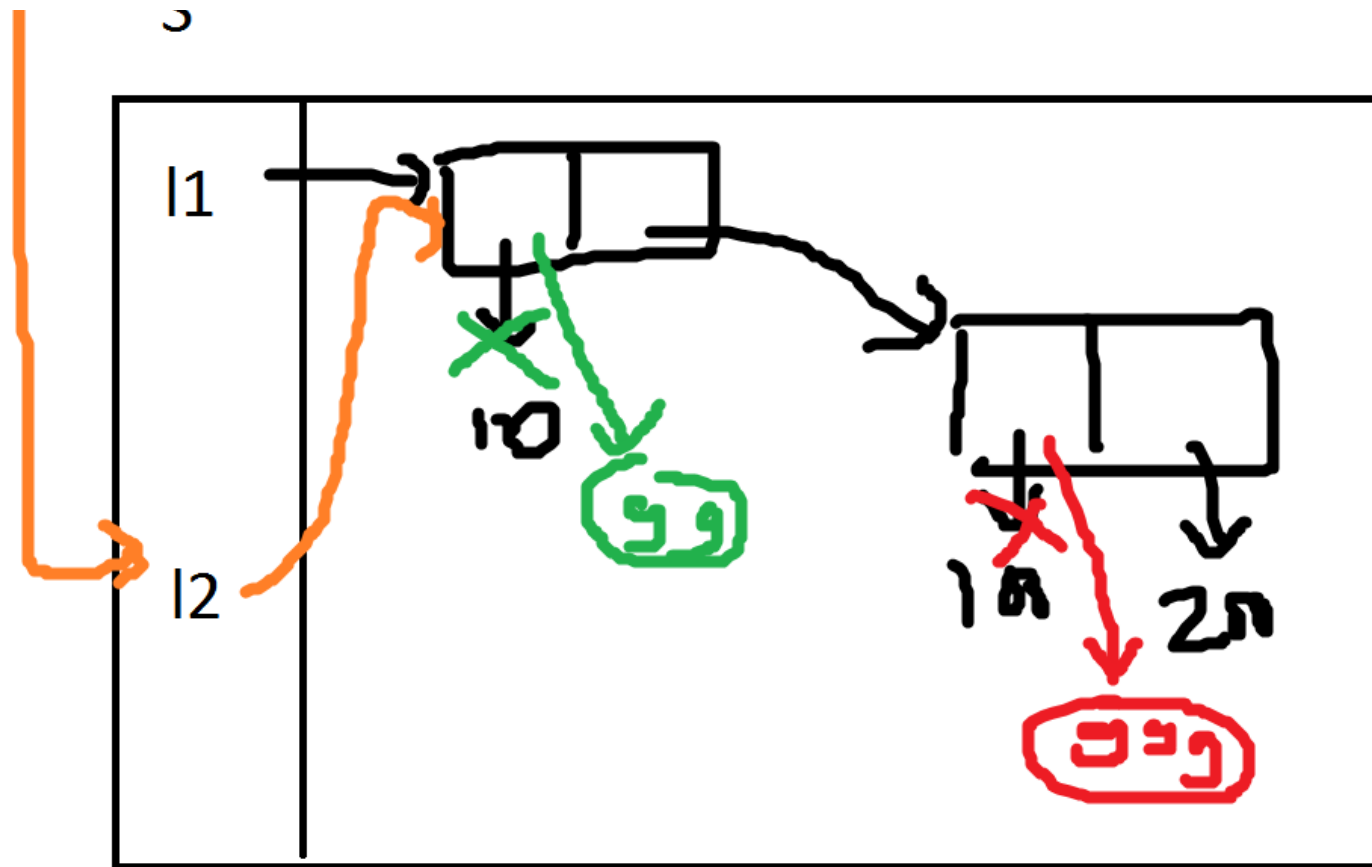
```
l1[0]=99
```

```
l1[1][0] = 999
```

c

H

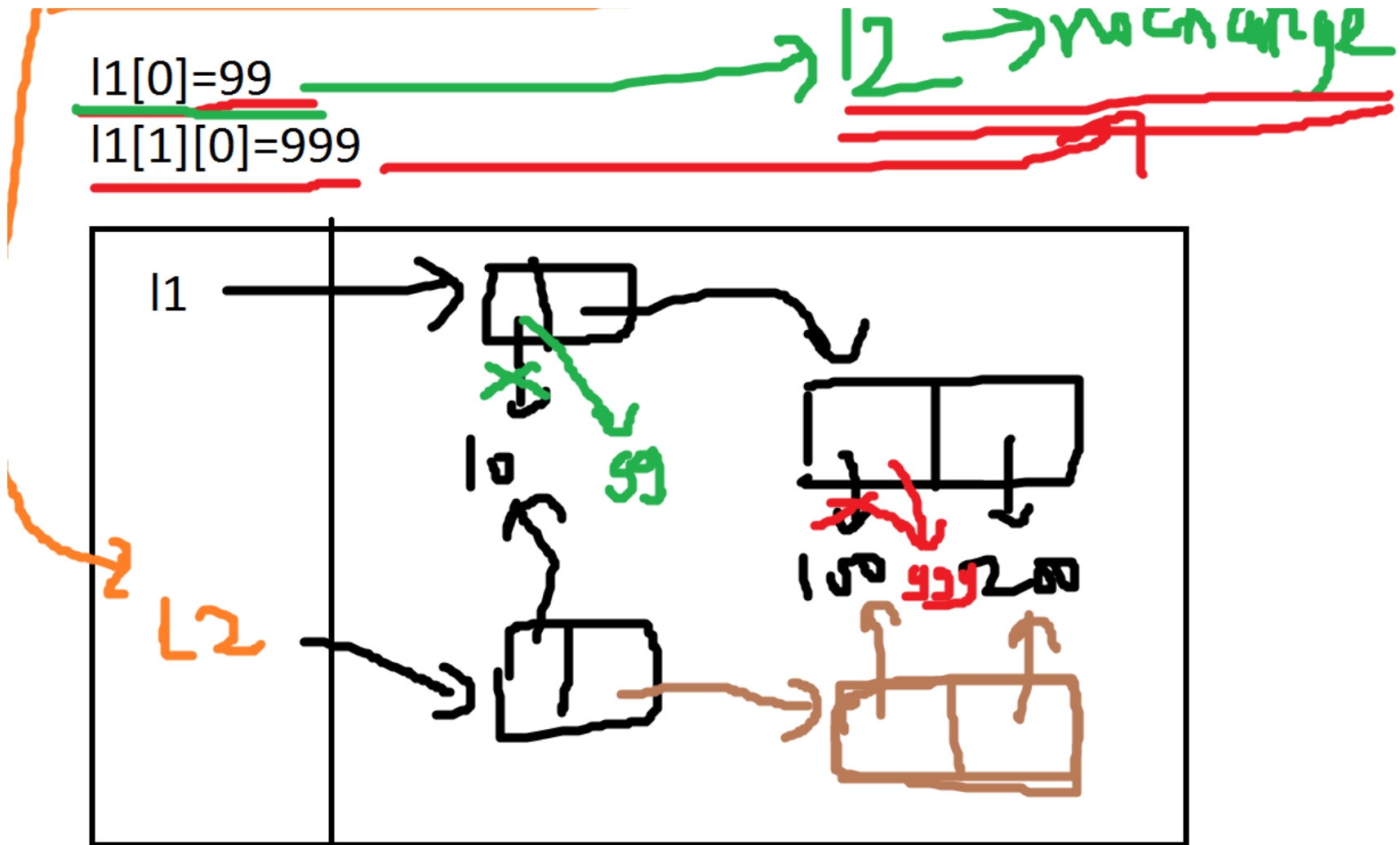
l2 affected



- ✓ Deep copy

```
l1 = [10,[100,200]]
```

```
import copy  
l2 = copy.deepcopy(l1)
```



1 Start coding or [generate](#) with AI.

Lexicographical ordering

- it is python's way to compare two sequences
- it is applicable to comparison of lists / strings / tuples

- it is applicable to comparison of lists / strings / tuples

- **Algorithm of lexicographical ordering**

1. take two strings (list / tuple)
2. compare first element of first string to first element of second string if same , repeat for the next element.
3. If all the elements of both strings are SAME --> then both strings are SAME ...

ex

```
s1="ABC"
```

```
s2="ABC"
```

Both are same !!

4. if two elements are not same ... then string with higher value of element is **greater** and other is **smaller**

ex

```
s1="ABC"
```

```
s2="AD"
```

A and A are same , but B and D are not same. SO string with D is greater ...

```
s1 < s2
```

ex

```
s1="QWERTY"
```

```
s2="QWERTZ"
```

$s1 < s2$ --> Y is smaller than Z

5. if all elements are same **BUT** one string finishes , then string with more no of characters is **greater**

ex

```
s1 = "QAZ"
```

```
s2 = "QA"
```

"QA" are match but s2 finishes ... so s1 is greater

```
s1 > s2
```

NOTE :: SAME algorithm works for list / string / tuple

Set Data Type

↳ 17 cells hidden

Frozenset Data type

↳ 4 cells hidden

✓ Dictionary Datatype

- it is ordered
- it stores key:value pairs
- key and value are given by the user
- using key first hash-value is calculated, and based on hash-value, element (key:value) pair is arranged
- key cannot be of mutable data type like list / set / dictionary
- value can be of any data type
- keys in dictionary cannot be duplicates

- dictionary is more memory efficient than set
- Ex

store employee id and employee info

store roll no and student info, student marks

store username and password

✓ NOTE :: SLICING is NOT ALLOWED in dictionary

```
1 #create dictionary of empid and empname
2 d1={1:"asdfg",2:"qwerty",3456:"yuio"}
3 print(d1)
4 print(d1.keys())
5 print(d1.values())
6 print(d1.items())
```

```
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}
dict_keys([1, 2, 3456])
dict_values(['asdfg', 'qwerty', 'yuio'])
dict_items([(1, 'asdfg'), (2, 'qwerty'), (3456, 'yuio')])
```

```
1 d1={1:"asdfg",2:"qwerty",3456:"yuio"}
2 print(d1)
3 #add new key value pair --> by direct assignment
4 d1[234]="bnm"
5 print(d1)
6 d1[12345]="xyz"
7 print(d1)
8 d1[999]="pqr"
9 print(d1) #dictionary maintains order of insertion for the items
```

```
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio', 234: 'bnm'}
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio', 234: 'bnm', 12345: 'xyz'}
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio', 234: 'bnm', 12345: 'xyz', 999: 'pqr'}
```


✓ Common operations on Dictionary

```

1 d1={1:"asdfg",2:"qwerty",3456:"yuio"}
2 print(d1)
3 #add new key value pair --> by direct assignment
4 d1[234]="bnm"
5 print(d1)
6 #modify a key-value pair --> by direct assignment
7 d1[234]="BMM"
8 print(d1)
9 #search a key in dictionary --> use in operator
10 k=2
11 if k in d1: #if k in d1.keys():
12     print(k, d1[k])
13 else:
14     print(k,"Not found in d1")
15 #print value for given key
16 print(d1[234])
17 #delete a key value pair --> del operator
18 del d1[234]
19 print("After deleting key value pair for 234 empid",d1)
20

```

```

{1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio', 234: 'bnm'}
{1: 'asdfg', 2: 'qwerty', 3456: 'yuio', 234: 'BMM'}
2 qwerty
BMM
After deleting key value pair for 234 empid {1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}

```

```

1 # print dictionary
2 d1={1:"asdfg",2:"qwerty",3456:"yuio"}
3 print(d1)
4 #print dictionary using for loop
5 for k in d1:

```

```

6  print(k,d1[k])#key and value
7  for k,v in d1.items():
8      print(k,v)#key and value
9

```

```

{1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}
1 asdfg
2 qwerty
3456 yuio
1 asdfg
2 qwerty
3456 yuio

```

```

1 # print dictionary where key is less than 3
2 d1={1:"asdfg",2:"qwerty",3456:"yuio"}
3 print(d1)
4 for k in d1:
5     if k < 3: # assuming all keys are numeric (int/float)
6         print(k,d1[k])

```

```

{1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}
1 asdfg
2 qwerty

```

```

1 #dictionary comprehension
2 # print dictionary where key is less than 3
3 d1={1:"asdfg",2:"qwerty",3456:"yuio"}
4 print(d1)
5 d2 = {k:v for k,v in d1.items() if k<3}
6 print(d2)

```

```

{1: 'asdfg', 2: 'qwerty', 3456: 'yuio'}
{1: 'asdfg', 2: 'qwerty'}

```

```

1 #sort the dictionary keys
2 d1={1:234,2:456,100:899,5:5,10:45,9:33}
3 print(d1)
4 print("sorted keys in d1 are")

```

```

4 print( sorted keys in d1 are )
5 print(sorted(d1.keys()))
6 print(sorted(d1))

```

```

{1: 234, 2: 456, 100: 899, 5: 5, 10: 45, 9: 33}
sorted keys in d1 are
[1, 2, 5, 9, 10, 100]
[1, 2, 5, 9, 10, 100]

```

```

1 #get all elements in a dictionary in reverse order
2 d1={1:234,2:456,100:899,5:5,10:45,9:33}
3 print(d1)
4 #reversed() function returns keys of dictionary in reverse order
5 for k in reversed(d1):
6     print(k,d1[k])

```

```

{1: 234, 2: 456, 100: 899, 5: 5, 10: 45, 9: 33}
9 33
10 45
5 5
100 899
2 456
1 234

```

```

1 #combine two dictionaries --> using | operator , update function
2 d1={1:234,2:456,100:899,5:5,10:45,9:33}
3 print(d1)
4 d2={123:567,23:67,5:566}
5 print(d2)
6 print(d1|d2)
7 print(d2|d1)
8 d3=d1|d2
9 print(d3)
10 d1.update(d2)
11 print(d1)

```

```

{1: 234, 2: 456, 100: 899, 5: 5, 10: 45, 9: 33}
{123: 567, 23: 67, 5: 566}
{1: 234, 2: 456, 100: 899, 5: 566, 10: 45, 9: 33, 123: 567, 23: 67}

```

```

{123: 567, 23: 67, 5: 5, 1: 234, 2: 456, 100: 899, 10: 45, 9: 33}
{1: 234, 2: 456, 100: 899, 5: 566, 10: 45, 9: 33, 123: 567, 23: 67}
{1: 234, 2: 456, 100: 899, 5: 566, 10: 45, 9: 33, 123: 567, 23: 67}

```

```

1 #unpacking of a dictionary
2 d1={1:234,2:456,100:899,5:5,10:45,9:33}
3 print(d1)
4 d2={123:567,23:67,5:566}
5 print(d2)
6
7 d3 = {**d1,**d2}
8 # ** operator unpacks the dictionary, then used for creating new dictionary
9 print(d3)
10 # DON'T use ** directly in print
11 print(**d1) #--> ERROR ( there are exceptions)

```

```

{1: 234, 2: 456, 100: 899, 5: 5, 10: 45, 9: 33}
{123: 567, 23: 67, 5: 566}
{1: 234, 2: 456, 100: 899, 5: 566, 10: 45, 9: 33, 123: 567, 23: 67}
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_8011/3332556438.py in <cell line: 0>()
      9 print(d3)
     10 # DON'T use ** directly in print
--> 11 print(**d1)

TypeError: keywords must be strings

```

how much is the max range of dictionary? can we store large reports?

- unlimited size
- we can store large reports
- PANDAS library uses dataframes which internally use dictionary

- ✓ which functions can we perform on dictionary if the key is in string type

```
1 #dictionary with key=name of student and value is marks
2 d1={"ss":40,"qq":89,"ui":95,"er":10}
3 #sort the keys --> using lexicographical ordering of keys
4 print(sorted(d1))
5 #extract one value using key
6 print(d1['qq'])
7 #update one value using key
8 d1['ui']=96
9 print(d1)
10 #almost operations like any dictionary
```

- ✓ Common simple questions with dictionary

```
1 # take a sentence /paragraph
2 # find frequency of each word in the sentence/paragraph
3 para="cat sat on the mat mat is for the cat sat mat cat rat rat"
4 #seperate all words / tokens --> list of words
5 # WRONG APPROACH
6 print(set(para)) # --> gives uniques characters and not words
7 # WRONG APPROACH
8 print(list(para)) # --> gives ALL characters and not words
9 #RIGHT APPROACH
10 word_list = para.split(" ")
11 print(word_list)
12 #unique words --> set on word_list
13 unique_words = set(word_list)
14 print(unique_words)
15
16 #Solution 1 -- efficient
17 word_count = {}
18 for word in word_list:
19     if word in word_count:
```

```

20     word_count[word] += 1 #increase frequency of word
21     else:
22         word_count[word] = 1 # add new word with freq 1
23 print(word_count)
24
25 #solution2 --> not very efficient
26 word_count = {}
27 unique_words = set(word_list)
28 for word in unique_words:
29     word_count[word] = word_list.count(word)
30
31 print(word_count)

```

```

{'c', 's', ' ', 'o', 't', 'h', 'e', 'm', 'f', 'i', 'r', 'a', 'n'}
['c', 'a', 't', ' ', 's', 'a', 't', ' ', 'o', 'n', ' ', 't', 'h', 'e', ' ', 'm', 'a', 't', ' ', 'm', 'a', 't', ' ', 'is', ' ', 'for', ' ', 'the', ' ', 'cat', ' ', 'sat', ' ', 'mat', ' ', 'cat', ' ', 'rat', ' ', 'rat']
{'cat', 'sat', 'on', 'the', 'mat', 'mat', 'is', 'for', 'the', 'cat', 'sat', 'mat', 'cat', 'rat', 'rat'}
{'cat', 'sat', 'mat', 'is', 'for', 'rat', 'the', 'on'}
{'cat': 3, 'sat': 2, 'on': 1, 'the': 2, 'mat': 3, 'is': 1, 'for': 1, 'rat': 2}
{'cat': 3, 'sat': 2, 'mat': 3, 'is': 1, 'for': 1, 'rat': 2, 'the': 2, 'on': 1}

```

```

1 """
2 20. Write a Python program to print all distinct values in a dictionary.
3 Sample Data : [{"V": "S001"}, {"V": "S002"}, {"VI": "S001"}, {"VI": "S005"},
4 {"VII": "S005"}, {"V": "S009"}, {"VIII": "S007"}]
5
6 Expected Output : Unique Values: {'S005', 'S002', 'S007', 'S001', 'S009'}
7 """
8 l_d = [{"V": "S001"}, {"V": "S002"}, {"VI": "S001"}, {"VI": "S005"},
9 {"VII": "S005"}, {"V": "S009"}, {"VIII": "S007"}]
10 unique_vals=[]
11 for d1 in l_d:
12     print(d1)
13     if len(d1) == 1:
14         val = list(d1.values())[0]
15         print(val)
16         if val not in unique_vals:
17             unique_vals.append(val)
18 print(unique_vals)

```

```

10 print(unique_rows,
{'V': 'S001'}
S001
{'V': 'S002'}
S002
{'VI': 'S001'}
S001
{'VI': 'S005'}
S005
{'VII': 'S005'}
S005
{'V': 'S009'}
S009
{'VIII': 'S007'}
S007
['S001', 'S002', 'S005', 'S009', 'S007']

```

✓ What is unhashable data type

- a data type whose hash value cannot be calculated using hash() function
- data type / object which contains mutable object(s) is UNHASHABLE

hash() function ONLY generates hash value for IMMUTABLE elements / objects!!

```

1 l1=[1,2,3]
2 print(hash(l1))

```

```

-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_8011/851437466.py in <cell line: 0>()
      1 l1=[1,2,3]
----> 2 print(hash(l1))

TypeError: unhashable type: 'list'

```

```
1 s1=set([1,2,3])
2 print(hash(s1))
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_8011/810526634.py in <cell line: 0>()
      1 s1=set([1,2,3])
----> 2 print(hash(s1))

TypeError: unhashable type: 'set'
```

```
1 t1=(1,2,3)
2 t2=(1,2,[100,200])
3 print(hash(t1)) #hashable
4 print(hash(t2)) #un-hashable
```

```
529344067295497451
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_8011/491338818.py in <cell line: 0>()
      2 t2=(1,2,[100,200])
      3 print(hash(t1)) #hashable
----> 4 print(hash(t2)) #un-hashable

TypeError: unhashable type: 'list'
```

```
1 Start coding or generate with AI.
```